

SUPSI

Proxy basato su Appwrite

Studente/i

**Mazza Luca
Mondini Manuela**

Relatore

Brocco Amos

Correlatore

-

Committente

Brocco Amos

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11160

Anno

2025 / 2026

Data

18 febbraio 2026

Indice

1 Abstract	1
2 Introduzione	2
2.1 Pianificazione	2
2.2 Requisiti	2
3 Stato dell'arte	3
4 Design e implementazione	4
4.1 API	4
5 Test e validazione	5
5.1 Risultati	5
6 Risultati	6
6.1 Manuale d'uso	6
7 Conclusioni	7
7.1 Sviluppi futuri	7
Glossario	I
Bibliografia	III

Elenco delle figure

Elenco delle tabelle

2.1	Requisiti del progetto	2
-----	----------------------------------	---

Capitolo 1

Abstract

Le policy di sicurezza di rete moderne spesso isolano sia client che server rendendo impraticabili le connessioni dirette senza l'utilizzo di complesse configurazioni. Questo progetto espone il design e lo sviluppo di un'**architettura client-server**, pensata per superare le limitazioni imposte da firewall e NAT nelle reti private. Il meccanismo centrale **sostituisce** le tradizionali **connessioni socket** con un approccio basato su **REST + WebSocket**, utilizzando Appwrite¹ [1] come Proxy. Il sistema crea un **canale** di comunicazione **sicuro** su cui client e server interagiscono asincronamente tramite eventi di creazione di documenti nel database di Appwrite.

Viene quindi sviluppata una **libreria modulare** in C++ (con QT [2]) per astrarre l'API di Appwrite, implementando funzionalità come la gestione delle **sessioni**, l'identificazione dei canali basata su **UUID** e la serializzazione dei payload in **JSON**. L'implementazione di tale libreria viene validata attraverso lo sviluppo di una semplice applicazione **Proof of Concept** (PoC), un **frontend**, realizzato con Qt/QML e compilato con WebAssembly [3] per l'esecuzione in browser, e un servizio **backend** nativo in C++.

Modern security policies often restrict direct client-server communication, especially in private networks. This project details the design and development of a **client-server architecture**, designed to overcome the limitations of firewalls and NAT in private networks. The core mechanism **replaces** the traditional client-server **socket connections** with a **REST + WebSocket** based approach, using Appwrite as a Proxy. The system creates a **secure** communication **channel** on which clients and servers interact asynchronously via document creation events in the Appwrite-hosted database.

A modular C++ (with QT) **library** is developed to abstract the Appwrite API, implementing features such as **session** management, **UUID**-based channel identification and **JSON** payload serialization. The implementation of said library is validated through the development of a simple **Proof of Concept** (PoC) application, a **frontend**, built with Qt/QML and compiled with WebAssembly for browser-based execution, and a native C++ **backend** service.

¹<https://appwrite.io>

Capitolo 2

Introduzione

Lo scopo del progetto è lo sviluppo di una libreria in C++ che consenta a client e server di comunicare in modo asincrono attraverso Appwrite ed un Proof of Concept che ne dimostri l'efficacia e l'utilità.

2.1 Pianificazione

1. **Analisi:**
2. **Applicazione demo:**
3. **Definire struttura API:**

2.2 Requisiti

ID	Descrizione	Note
R-01	<i>Proxy</i> deve supportare l'API di Appwrite	-
R-02	X	
R-03	X	
R-04	X	
R-05	X	
R-06	X	
R-07	X	
R-08	X	
R-09	X	

Tabella 2.1: Requisiti del progetto

Capitolo 3

Stato dell'arte

Capitolo 4

Design e implementazione

4.1 API

Di seguito è riportato un esempio di codice per l'inizializzazione.

```
// Esempio di inizializzazione
float x = 0.5f;
std::vector<float> input = {x, x, x}; // Input per il modello
// Altri parametri...
```

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

API Application Programming Interface. Un insieme di definizioni e protocolli utilizzati per costruire e integrare il software applicativo. 1

JSON JavaScript Object Notation. Un formato di testo standard aperto, leggero e leggibile dall'uomo, utilizzato per lo scambio di dati tra client e server. 1

NAT Network Address Translation. Una tecnica usata dai router per modificare gli indirizzi IP dei pacchetti in transito, permettendo a più dispositivi di condividere un singolo indirizzo IP pubblico. 1

Proxy Un componente software o hardware che funge da intermediario tra un client e un server, inoltrando le richieste e le relative risposte. 1

UUID Universally Unique Identifier. Un identificativo standardizzato a 128 bit utilizzato nei sistemi informatici per identificare in modo univoco informazioni e risorse. 1

WebAssembly Un formato di istruzioni binario portabile, progettato per consentire l'esecuzione di codice ad alte prestazioni all'interno dei browser web. 1

Bibliografia

- [1] Appwrite Team. *Appwrite Documentation - Open-Source Backend Server*. 2026. URL: <https://appwrite.io/docs> (visitato il 18/02/2026).
- [2] The Qt Company. *Qt 6 Documentation*. 2026. URL: <https://doc.qt.io/qt-6/> (visitato il 18/02/2026).
- [3] WebAssembly Community Group. *WebAssembly Specification*. 2026. URL: <https://webassembly.org/> (visitato il 18/02/2026).